

Deployment Guide — Digital Revolution Job Tracker

****Target:**** Ubuntu Server 26.04 (fresh install)

****Access:**** Custom domain with HTTPS

****Source:**** Git clone from GitHub

Prerequisites

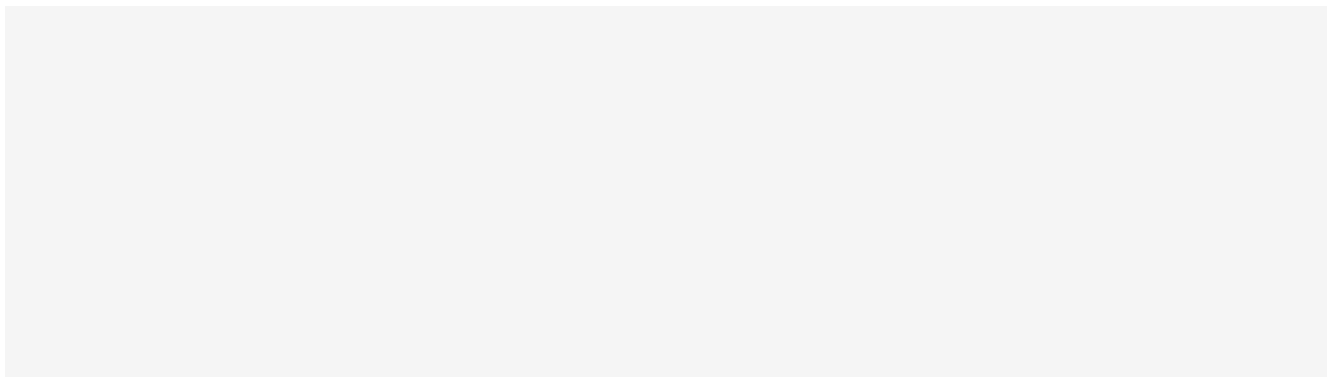
Before you start, make sure you have:

- A domain name pointing to your server's public IP (A record, e.g. `jobs.yourdomain.com`)
 - Your GitHub repo is up to date:

```
```bash
git push origin arena/019fba35-customer-job-app
```
```
 - SSH access to your Ubuntu server
-

Step 1 — Initial Server Setup

SSH into your server and run the basics:



Firewall — allow SSH, HTTP, HTTPS

Step 2 — Install Node.js 22 LTS

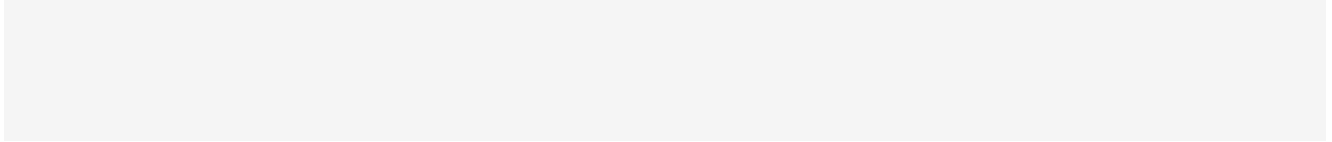
Step 3 — Install PostgreSQL

Create database and user

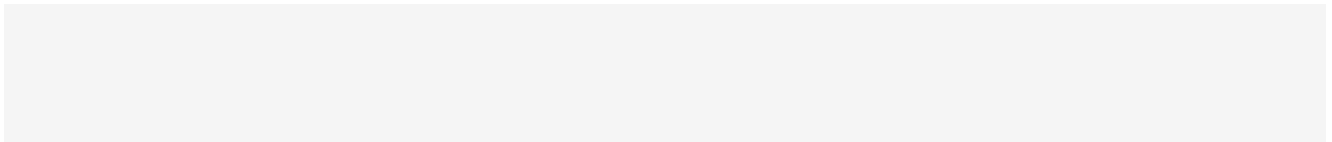
Run these SQL commands inside the psql prompt:

& p ****Replace `CHANGE\_THIS\_TO\_A\_STRONG\_PASSWORD`** with a strong, unique password.**

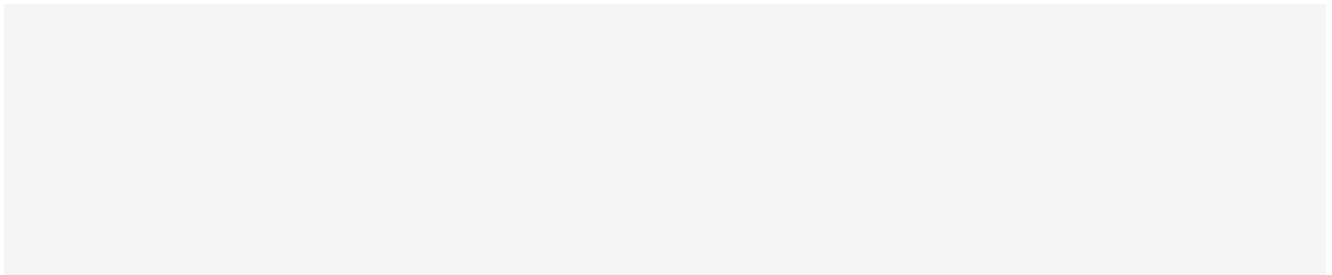
Step 4 — Clone the Repository



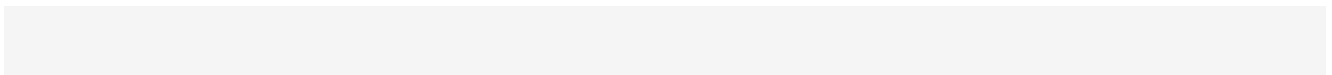
Step 5 — Configure Environment Variables



Edit the file to match your setup:

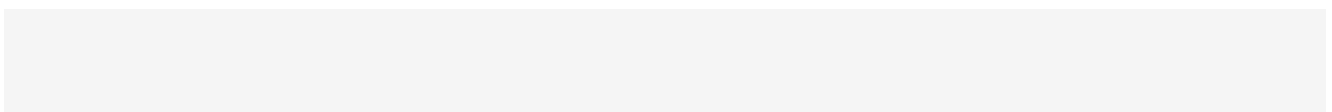


Generate a secure JWT secret:



& p Replace `YOUR\_STRONG\_PASSWORD`, the JWT secret, domain URL, and admin credentials with your own values.

Step 6 — Install Dependencies & Set Up Database



Generate and push the database schema with Drizzle

If ``drizzle-kit migrate`` doesn't apply (it looks for a migrations folder), you can alternatively use ``npx drizzle-kit push`` which syncs the schema directly.

Step 7 — Build the Application

This creates the optimized production build in `[.next/]`.

Step 8 — Set Up PM2 (Process Manager)

PM2 keeps your app running and auto-restarts it on crashes or server reboots.

Useful PM2 commands

Step 9 — Install & Configure Nginx (Reverse Proxy)

Create a site config:

Paste this configuration (replace [jobs.yourdomain.com] with your domain):

Enable the site and test:

At this point, you can test that the app is working over HTTP at [http://jobs.yourdomain.com].

Step 10 — Set Up HTTPS with Let's Encrypt

Certbot automatically modifies your Nginx config to handle HTTPS and sets up a cron/timer for auto-renewal.

Step 11 — Verify Everything Works

1. Visit `https://jobs.yourdomain.com` in your browser
 2. Log in with the default admin credentials from your `.env`` file
 3. Test adding a customer and a job
 4. Check PM2 logs for any errors: `pm2 logs jobtracker``
-

Updating the App (Future)

Whenever you push changes to GitHub:

Security Checklist

- Firewall (UFW) enabled — only ports 22, 80, 443 open
- Strong database password set
- JWT secret is a random 64+ byte hex string
- Admin password changed from default
- HTTPS enabled with Let's Encrypt
- Non-root user running the app
- (Optional) Set up fail2ban for SSH brute-force protection:
``bash

```
sudo apt install -y fail2ban
sudo systemctl enable fail2ban
```
```

- (Optional) Set up automated database backups:

```
```bash
# Add to crontab (crontab -e):
# 0 2 * * * pg_dump -U jobtracker jobtracker_db > /home/deploy/backups/jobtracker_$(date +%Y\
%m\%d).sql
```
```

## File Structure on Server

## Troubleshooting

| App won't start            | Check <code>pm2 logs jobtracker</code> for errors                                          |
|----------------------------|--------------------------------------------------------------------------------------------|
| Database connection error  | Verify <code>DATABASE_URL</code> in <code>.env</code> matches your PostgreSQL credentials  |
| 502 Bad Gateway            | Ensure PM2 is running ( <code>pm2 status</code> ) and Nginx proxy is pointing to port 3000 |
| Domain not resolving       | Check DNS A record points to your server's public IP                                       |
| SSL certificate error      | Run <code>sudo certbot renew --force-renewal</code>                                        |
| Permission denied on clone | Use HTTPS URL or set up SSH keys for GitHub                                                |
| Drizzle migration fails    | Ensure the PostgreSQL user has GRANT ALL on the database and schema                        |

















